Laporan Praktikum 6

Modul 6: Protokol Komunikasi Telemetri Industri (MQTT)

Mata Kuliah Teknologi IoT II

Oleh :

Nama : Sofy Salsabila

NPM : 25782084

Kelas : TRI – 3C

Dosen Pengampu :

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T., IPM. ASEAN Eng.

Agus Ambarwari, S.Pd, M.Kom

PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2025/2026

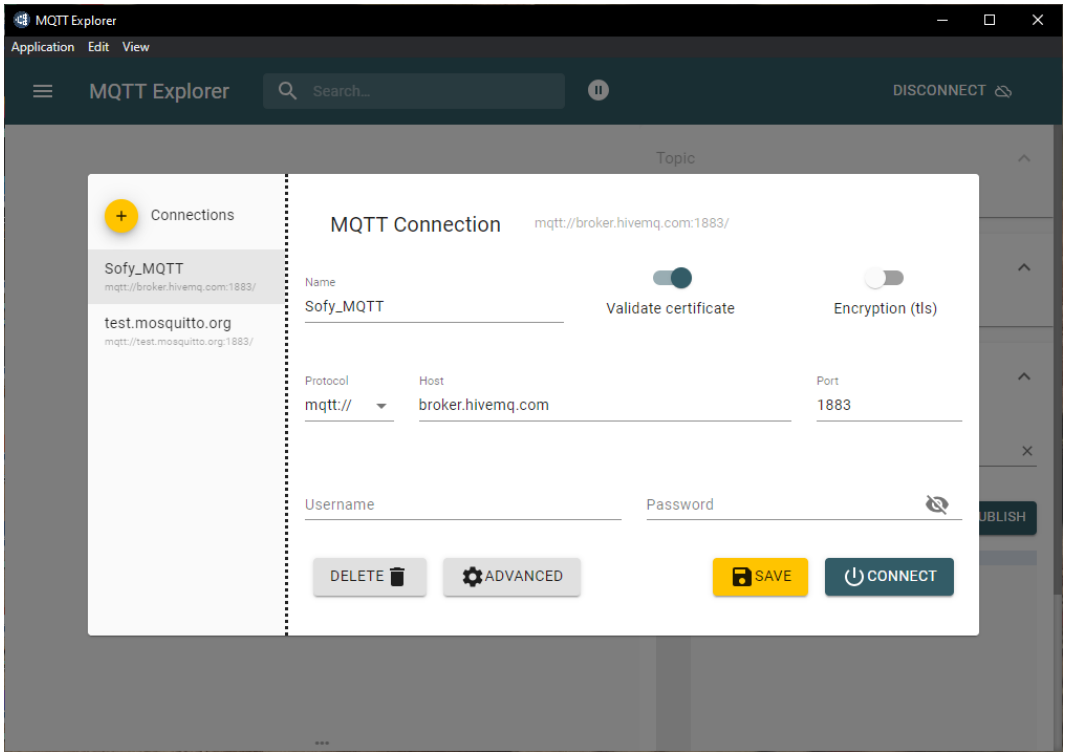
Modul 6: Protokol Komunikasi Telemetri Industri (MQTT)

Jadi, yang pertama-tama sebelum melakukan praktikum saya mencari dan instal pustaka di Arduino **PubSubClient** oleh Nick O'Leary, **ArduinoJson** oleh Benoit Blanchon, dan **DHT sensor library** oleh Adafruit. Kemudian, download alat uji telemetri mandiri, yaitu **MQTT Explorer**

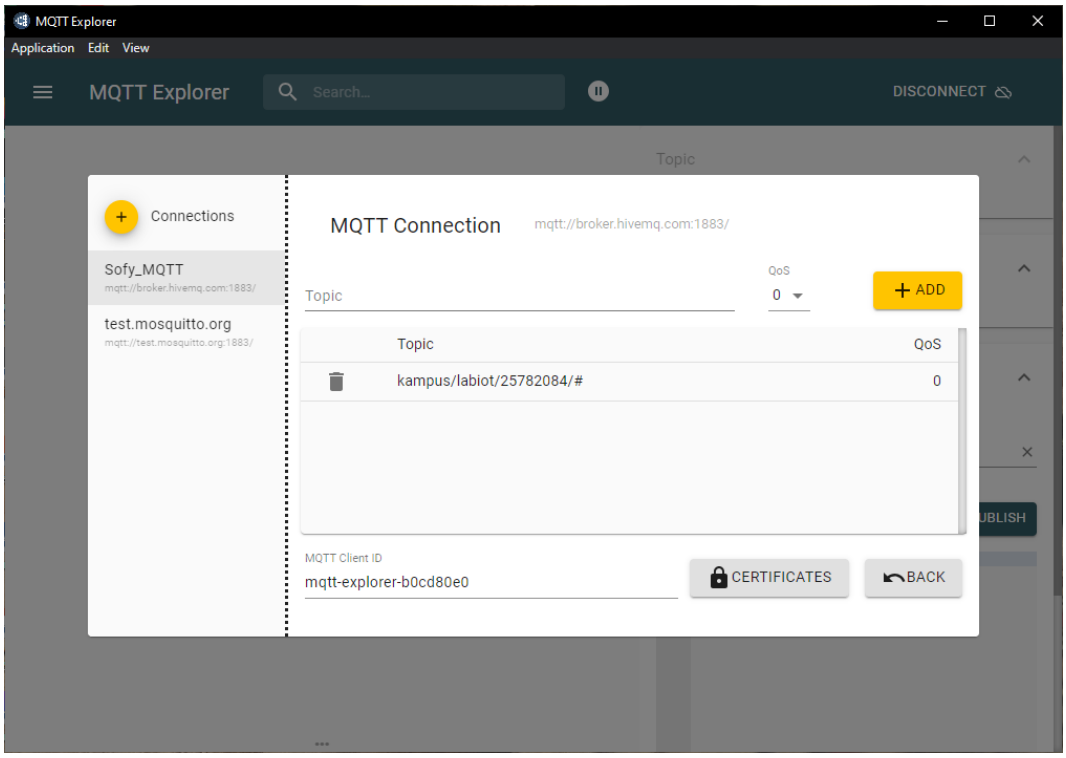
Praktikum

Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

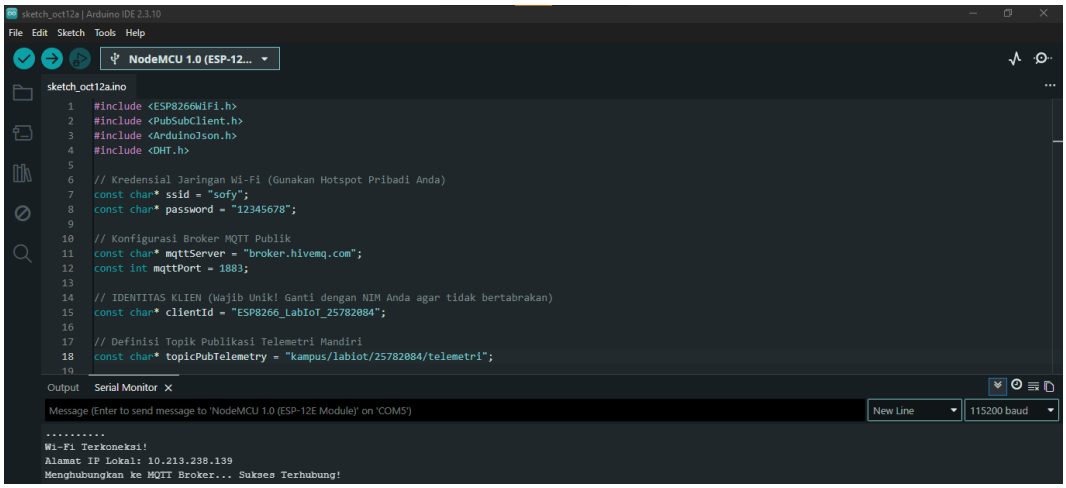
Disini saya mengkonfigurasi koneksi di MQTT Explorer. Di sini saya menentukan alamat server (Host) yang digunakan yaitu broker publik broker.hivemq.com pada port 1883 tanpa enkripsi TLS (standar port MQTT teks terbuka). Nama profil koneksi saya kasih nama saya Sofy_MQTT.



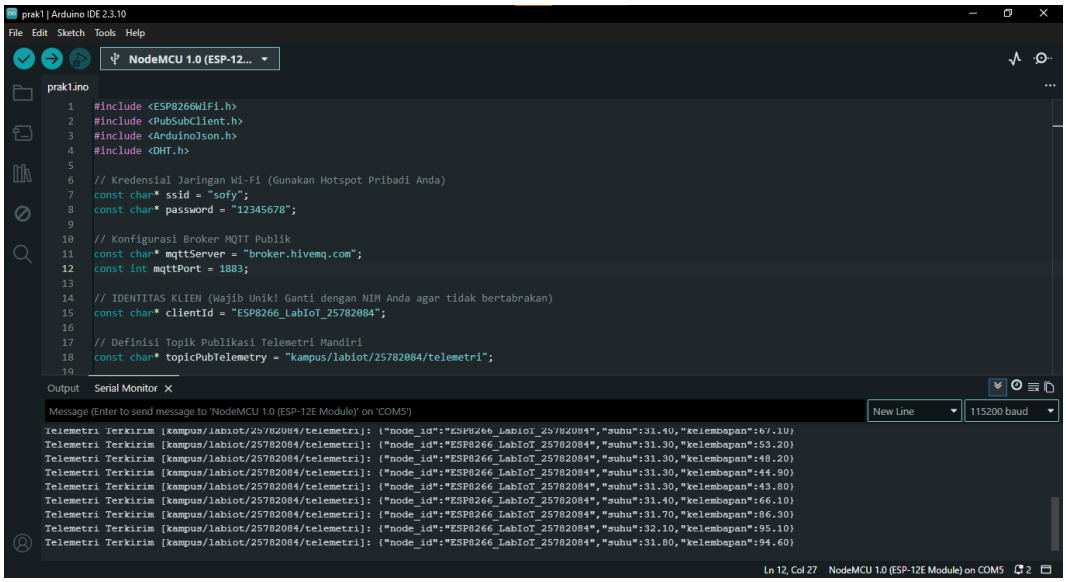
Ini saya masukin topic nya di pengaturan menu Advanced/Topic. Saya disini masukin topik kampus/labiot/25782084/# menggunakan multi-level wildcard (#) agar hanya memantau lalu lintas data milik NPM saya sendiri (25782084).



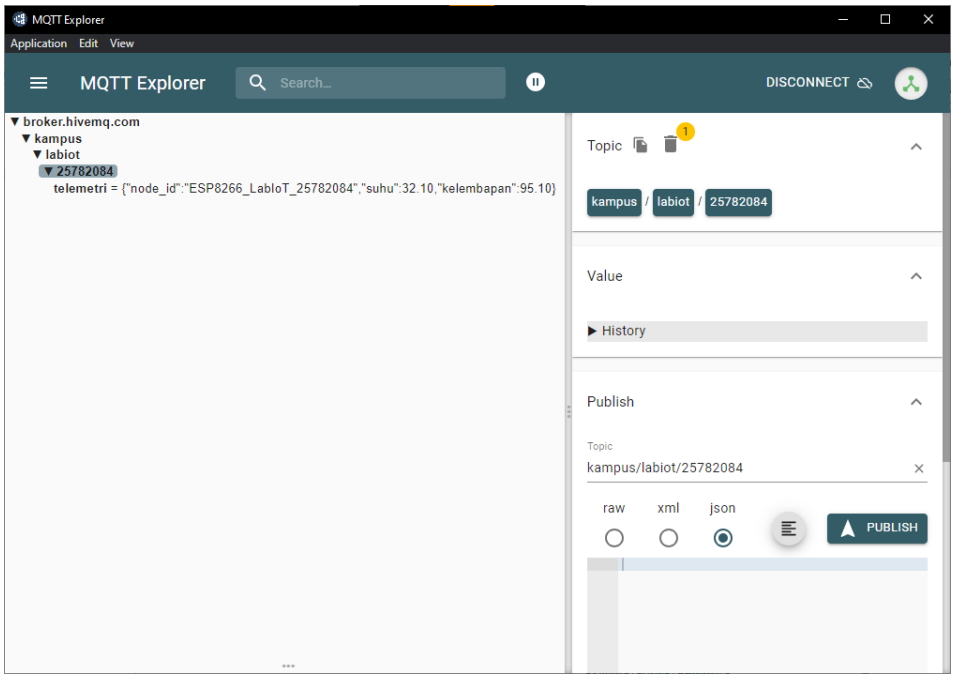
Jadi disini kodenya sudah saya sesuaikan semuanya, kemudian saya run dan jalankan. NodeMCU berhasil terhubung ke jaringan Wi-Fi sofy, mendapatkan alamat IP lokal 10.213.238.139, lalu berhasil terhubung dan terhubung ke broker MQTT (broker.hivemq.com).



Lalu setelah itu, di Serial Monitor memperlihatkan log pengiriman data dari NodeMCU yang mencetak pesan Telemetri Terkirim secara berkala setiap 5 detik ke topik kampus/labiot/25782084/telemetri, dimana sistem berhasil menyusun data pembacaan sensor DHT yaitu nilai desimal suhu 32.10 °C dan kelembapan 95.10% dengan identitas perangkat ESP8266_LabIoT_25782084 ke dalam format JSON yang valid.



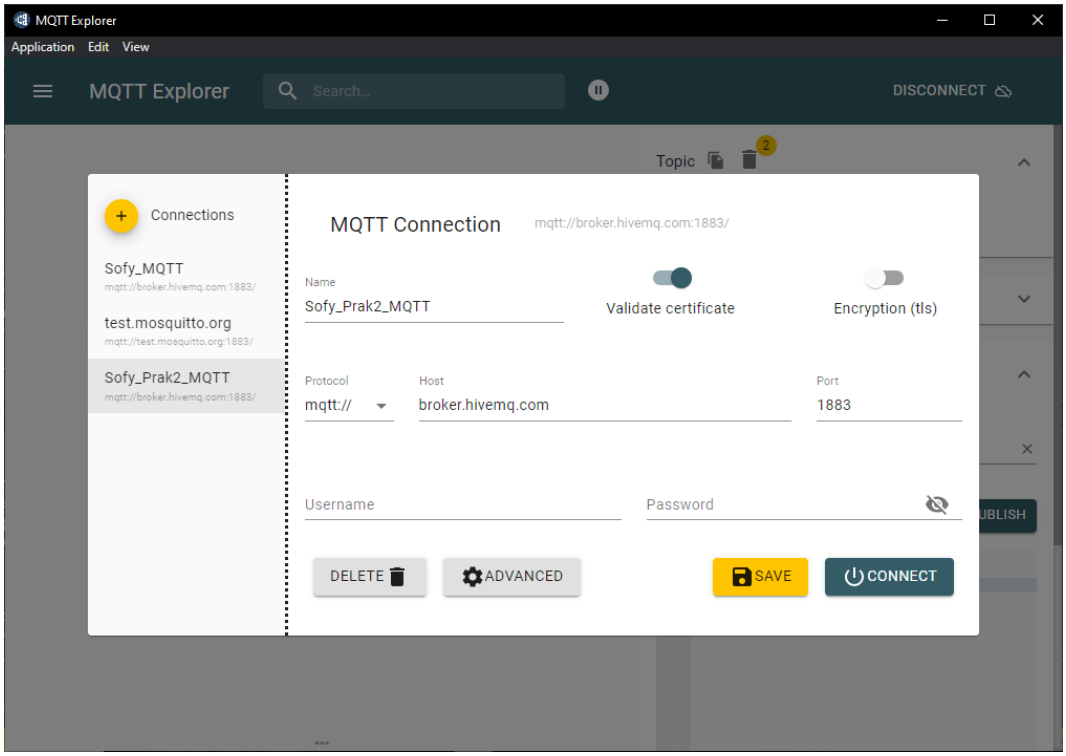
Kemudian di MQTT Explorer berhasil menerima paket data secara real-time di sisi cloud broker, dimana struktur hierarki topik kampus/labiot/25782084/telemetri terbentuk otomatis di panel Topic Tree dan menampilkan payload JSON berisi pembacaan suhu 32.10 °C serta kelembapan 95.10% yang nilainya sinkron dengan yang dikirim tadi.



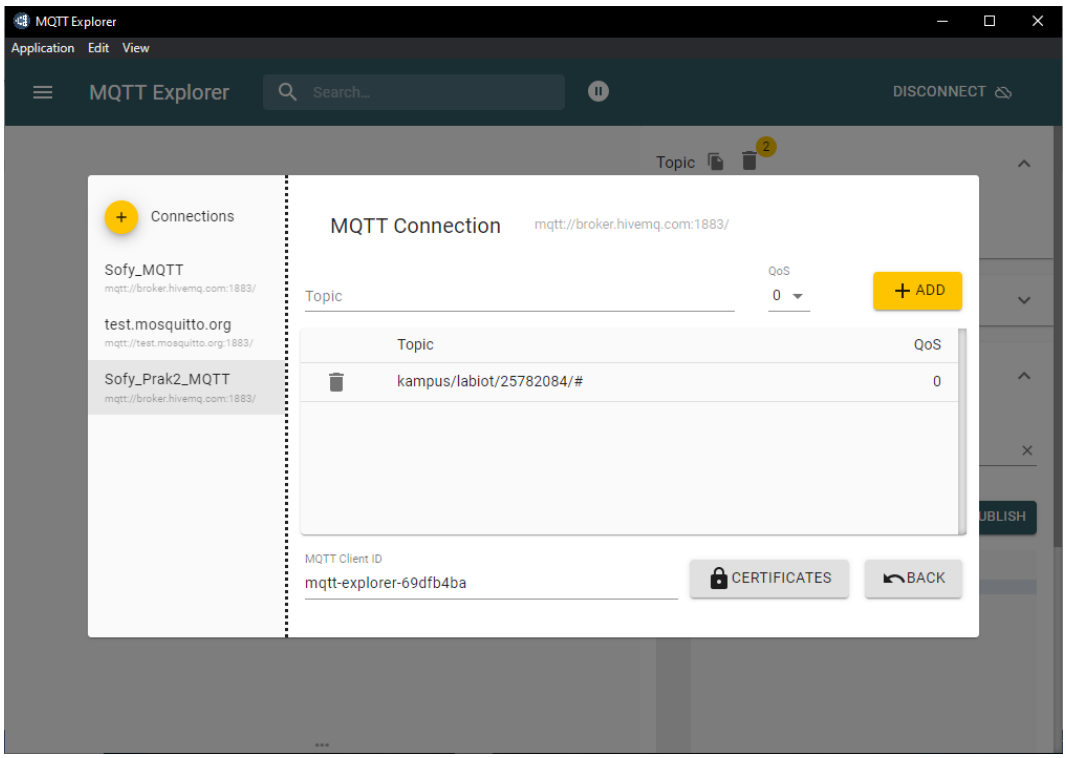
Jadi pada Praktikum 1 ini, saya menggunakan NodeMCU ESP8266 sebagai Publisher (pengirim data) utama yang terhubung ke jaringan Wi-Fi. Perangkat ini menggunakan sensor DHT22 untuk membaca data suhu dan kelembapan secara real-time. Selanjutnya, data tersebut dikemas oleh NodeMCU ke dalam format JSON dan dikirimkan secara otomatis setiap 5 detik ke broker publik broker.hivemq.com melalui topik khusus kampus/labiot/25782084/telemetri. Hasilnya, data suhu dan kelembapan berhasil terkirim dari Publisher dan tampil secara real-time di Subscriber dengan isi paket JSON yang sinkron antara pengirim dan penerimanya.

Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

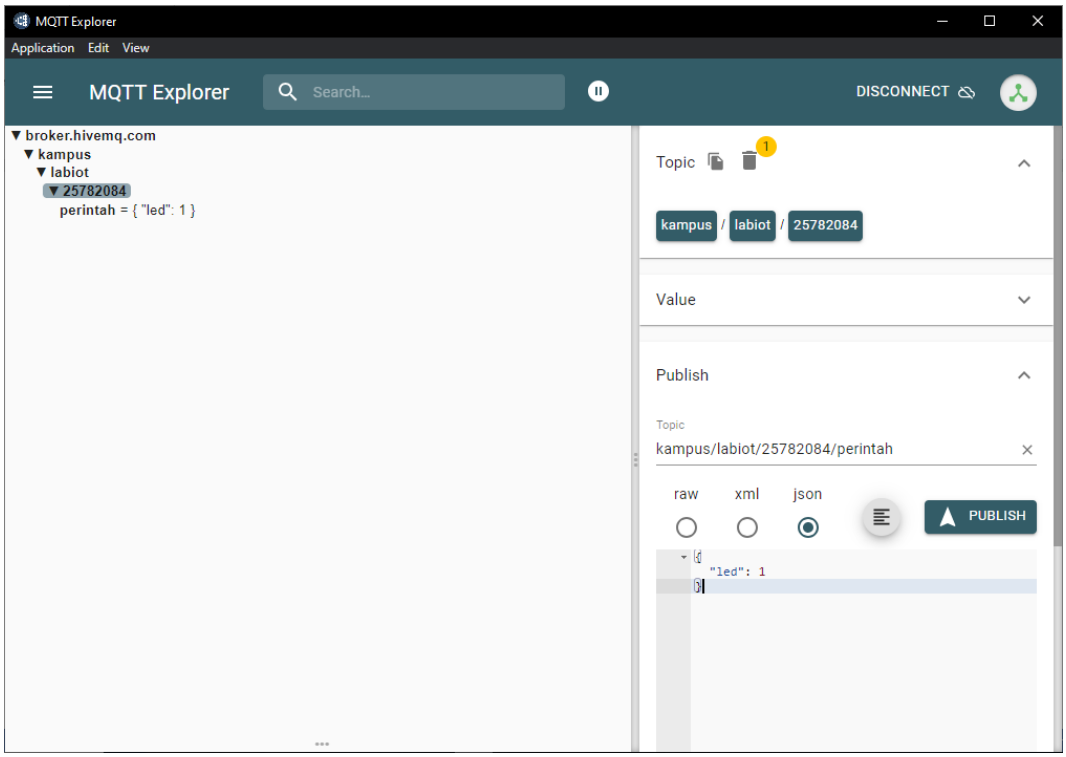
Konfigurasi Broker di MQTT Explorer



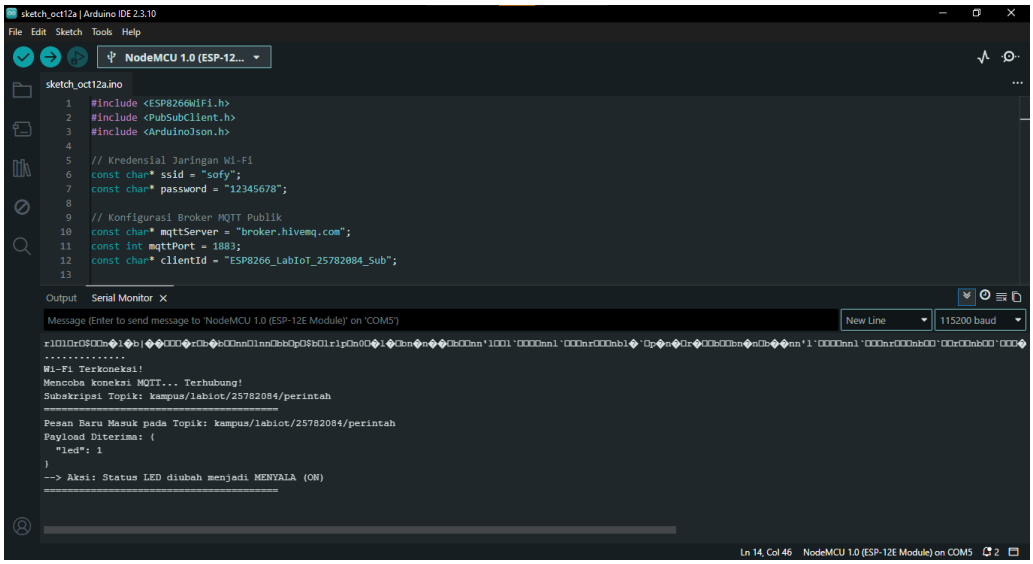
Membuat topic



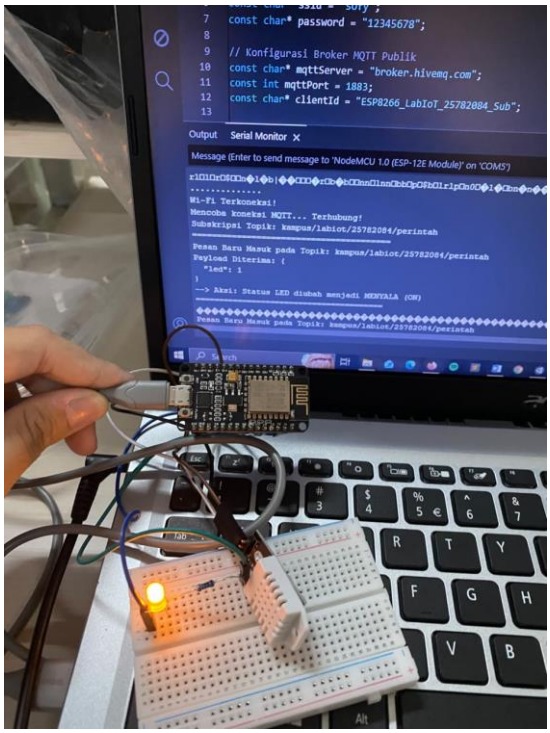
Jadi, di MQTT Explorer sebagai Publisher (pengirim perintah). Saya menentukan topik perintah spesifik yaitu kampus/labiot/25782084/perintah, memformat jenis payload ke bentuk JSON, lalu memasukkan teks perintah {"led": 1}. Setelah tombol Publish ditekan, broker HiveMQ secara real-time membentuk hierarki topik di panel Topic Tree dan menyalurkan paket perintah tersebut.



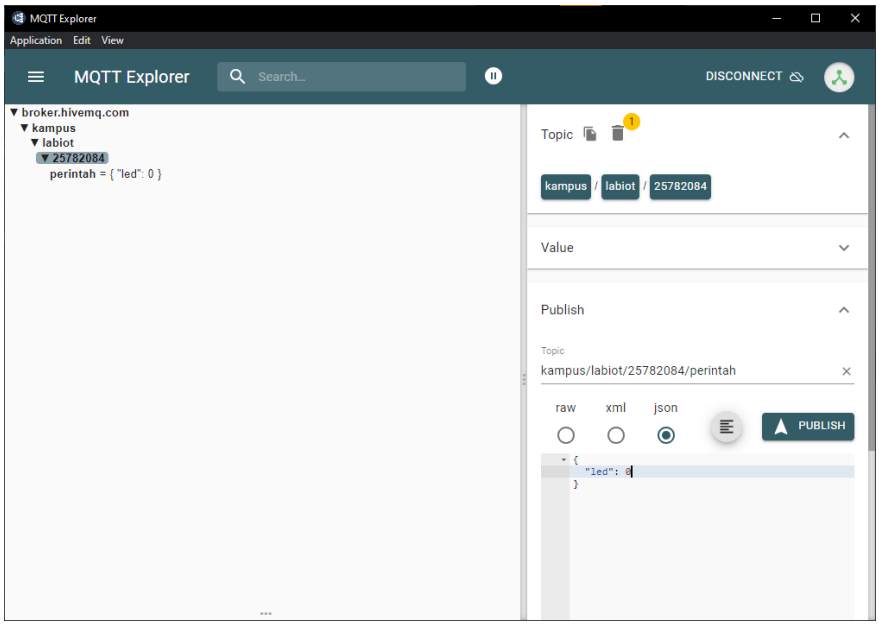
Serial Monitor Arduino IDE yang terhubung ke NodeMCU ESP8266 sebagai Subscriber (penerima data). Jadi, NodeMCU berhasil tersambung ke jaringan Wi-Fi sofy, terhubung ke broker HiveMQ, dan mendaftarkan subskripsi pada topik kampus/labiot/25782084/perintah menggunakan clientId ESP8266_LabIoT_25782084_Sub. jadi, Ketika paket dari MQTT Explorer masuk, fungsi callback membedah payload {"led": 1} dan mencetak konfirmasi: --> Aksi: Status LED diubah menjadi MENYALA (ON).



Jadi ini adalah LED yang sudah menyala saat sinyal perintah {"led": 1} berhasil diproses oleh NodeMCU.



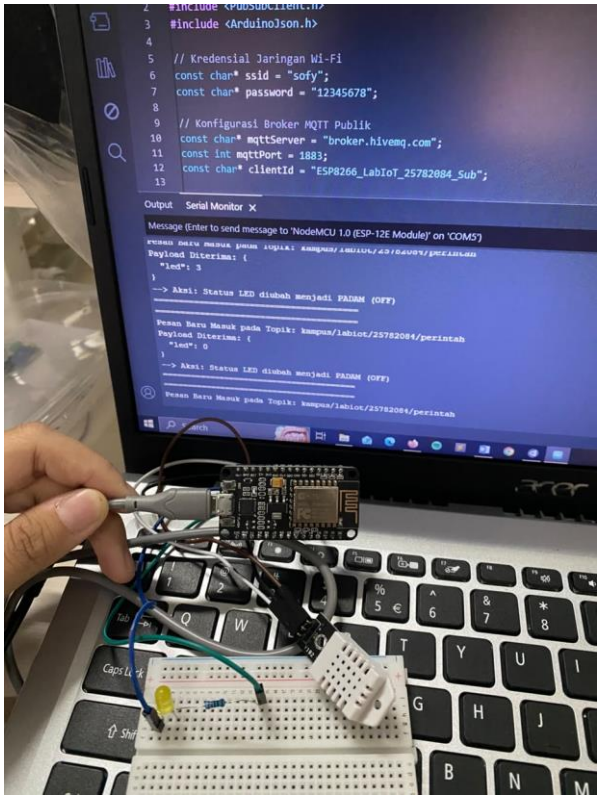
Pada Publish MQTT Explorer, saya merubah teks payload JSON menjadi {"led": 0} pada topik kampus/labiot/25782084/perintah. Pada program NodeMCU, sistem diatur dengan logika perbandingan bahwa hanya nilai 1 yang dianggap sebagai sinyal untuk menyalakan LED. Jadi, pengiriman nilai 0 atau angka selain 1 akan diidentifikasi oleh sistem sebagai perintah untuk mematikan LED.



Pada Serial Monitor, fungsi *callback* menangkap *payload* {"led": 0} dan mencetak konfirmasi: --> Aksi: Status LED diubah menjadi PADAM (OFF).



LED langsung padam membuktikan bahwa pengiriman sinyal selain 1 berhasil mematikan aliran arus listrik ke pin LED.

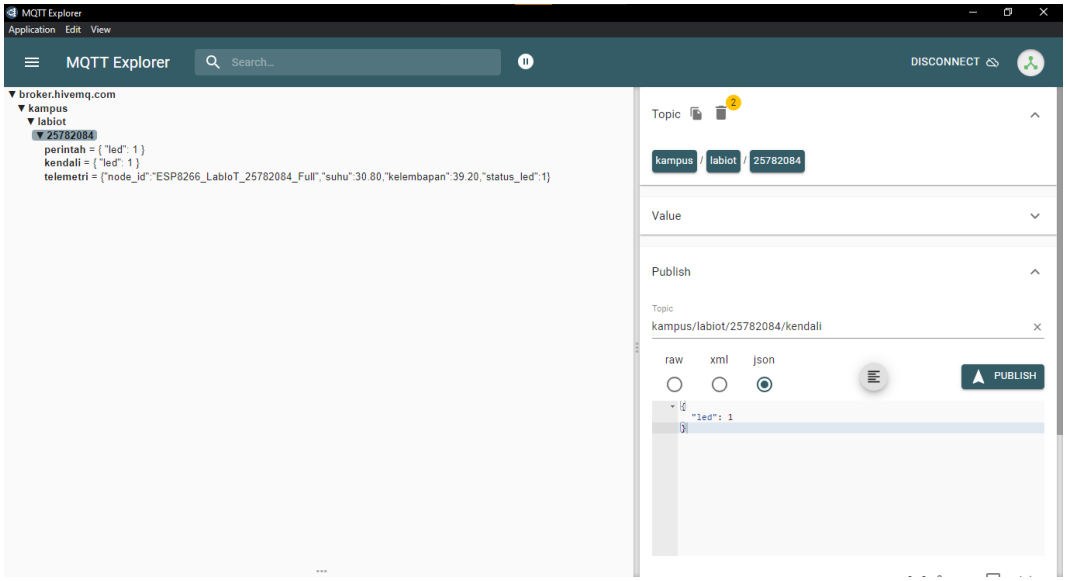


Pada Praktikum 2 ini, disini saya melakukan komunikasi MQTT dua arah untuk kendali aktuator asinkron. MQTT Explorer sebagai Publisher yang mengirimkan instruksi JSON melalui topik kampus/labiot/25782084/perintah ke broker publik broker.hivemq.com. Sebaliknya, NodeMCU ESP8266 sebagai Subscriber yang mendaftarkan diri pada topik tersebut menggunakan clientId ESP8266_LabIoT_25782084_Sub.

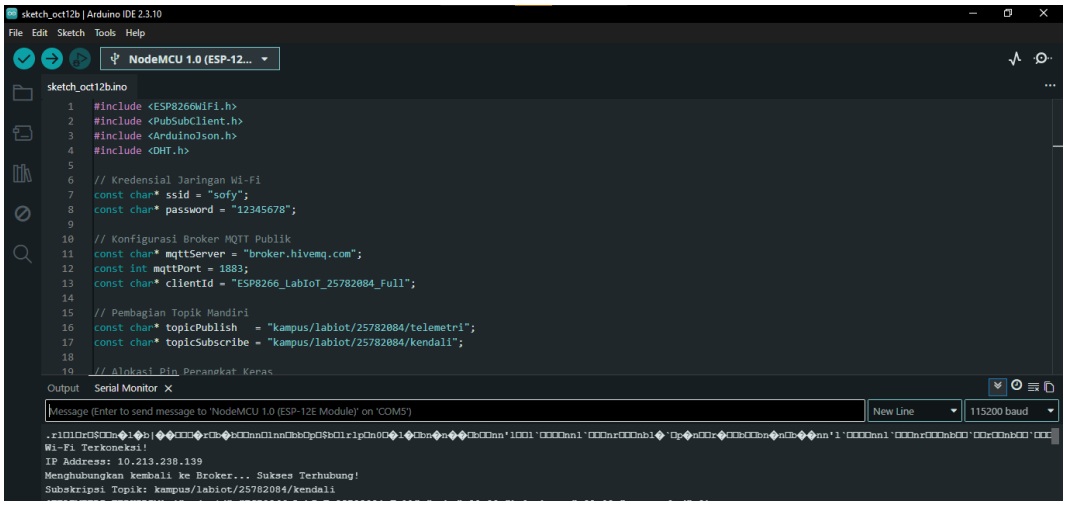
Jadi, pengiriman payload {"led": 1} berhasil memicu fungsi callback untuk menyalakan LED fisik, sedangkan pengiriman payload {"led": 0} (maupun angka selain 1) berhasil memadamkan LED fisik secara instan, dengan status log Serial Monitor yang terbukti sinkron dengan kondisi LED.

Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)

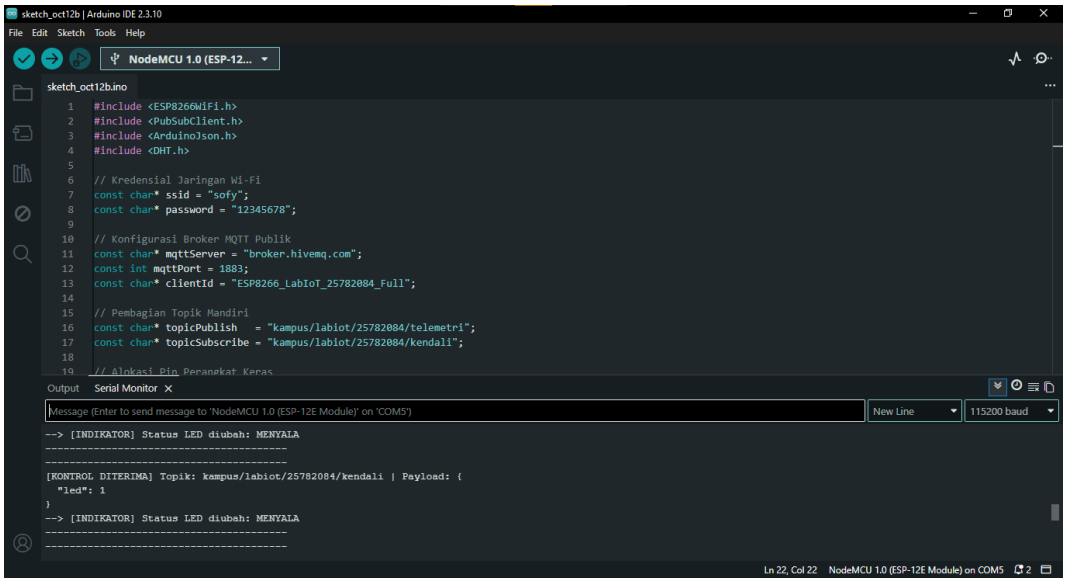
MQTT Explorer berhasil terhubung ke broker broker.hivemq.com. Pada Topic Tree, terbentuk hierarki topik kampus/labiot/25782084 yang memuat topik telemetri (menerima data sensor) serta topik kendali.



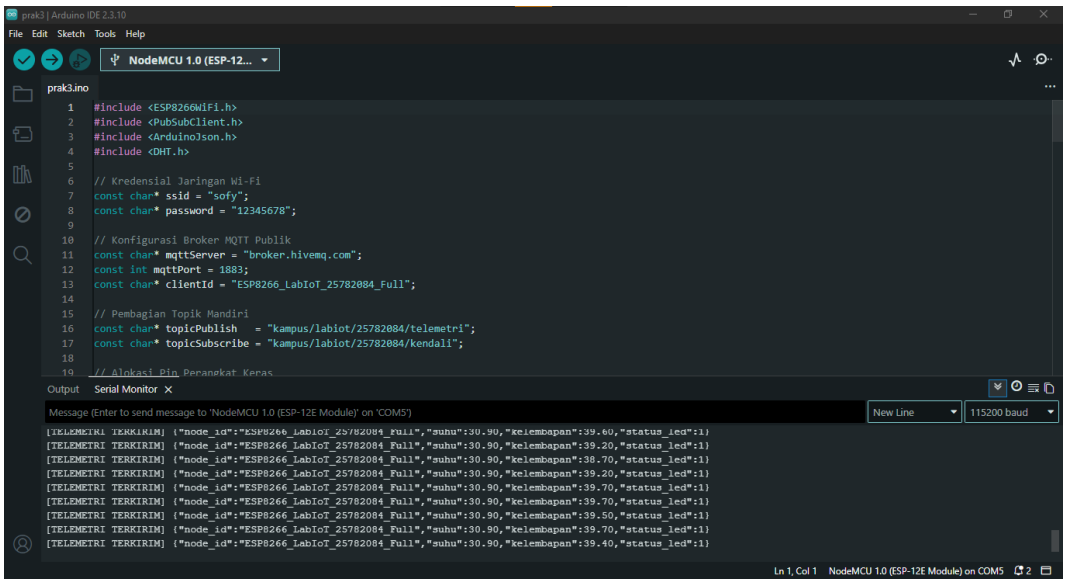
NodeMCU terhubung ke Wi-Fi sofy dengan IP 10.213.238.139, berhasil tersambung ke broker, serta mengonfirmasi subskripsi pada topik kampus/labiot/25782084/kendali.



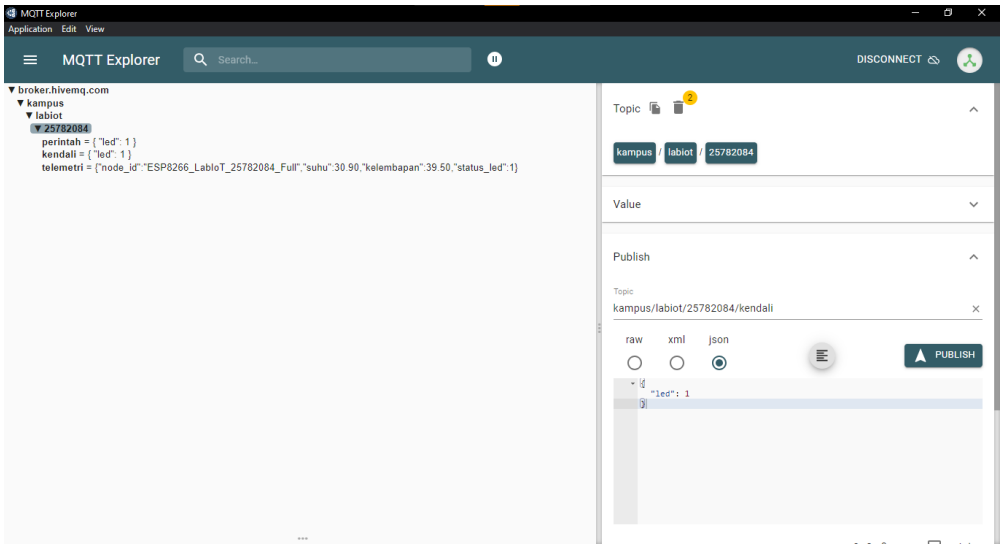
Tampilan Serial Monitor jadi NodeMCU menerima payload perintah {"led": 1} dari topik kendali. Fungsi callback langsung memproses data tersebut dan mencetak konfirmasi --> [INDIKATOR] Status LED diubah: MENYALA.



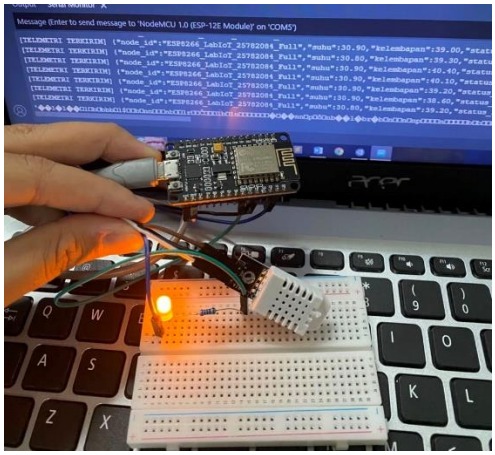
Menampilkan log pengiriman data telemetri secara setiap 4 detik. NodeMCU memublikasikan paket JSON berisi ID perangkat, nilai suhu, kelembapan, serta status LED fisik (status_led: 1) ke topik kampus/labiot/25782084/telemetri.



Disini menunjukkan MQTT Explorer yang menerima pembaruan payload JSON telemetri dari NodeMCU secara real-time (suhu: 30.90, kelembapan: 39.50, status_led: 1).



LED yang menyala



Pada Praktikum 3 ini, saya melakukan sistem komunikasi MQTT terpadu dua arah (Full-Duplex Telemetry & Control). NodeMCU ESP8266 dengan clientId ESP8266_LabIoT_25782084_Full sebagai Publisher yang memublikasikan data telemetri (suhu, kelembapan, dan status LED) ke topik kampus/labiot/25782084/telemetri setiap 4 detik, sekaligus sebagai Subscriber yang mendengarkan topik kampus/labiot/25782084/kendali. Lalu, MQTT Explorer bertindak Subscriber pemantau data telemetri serta sebagai Publisher yang mengirimkan paket perintah {"led": 1}. Pengujian membuktikan bahwa proses pengiriman telemetri sensor dan penerimaan instruksi kendali LED dapat berjalan secara full-duplex tanpa saling memblokir (non-blocking), serta respon LED fisik sinkron yaitu menyala.

Latihan

Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):

Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:

1. Efisiensi penggunaan kapasitas *bandwidth* pada paket data seluler.
   Jawab : Di HTTP, setiap kali mau ngirim data sensor meskipun isinya cuma teks JSON kecil, wajib nempel-in header bawaan HTTP yang ukurannya lumayan gede, sekitar 200–800 byte. Isinya macam-macam, seperti data browser, tipe konten, dan informasi server.
   Sedangkan di MQTT, header ringkas dan ringan, cuma butuh minimal 2 byte aja. Karena tidak perlu bawa header teks yang berat setiap kali ngirim data, transmisi paket data seluler jadi jauh lebih hemat dan ramah kuota.

2. Beban komputasi mikrokontroler (*CPU overhead*) saat menangani pertukaran data dua arah.
   Jawab : HTTP itu sifatnya Request-Response dan koneksinya langsung putus. Jadi setiap kali ESP8266 mau ngirim data atau nunggu perintah baru, dia harus bolak-balik buka-tutup koneksi jaringan (proses handshake TCP) dari awal. Kalau kita mau bikin kontrol yang real-time, ESP8266 harus terus-terusan minta data (polling), bikin pemrosesan CPU dan RAM-nya kerja keras.
   Kalau di MQTT, ESP8266 cuma perlu melakukan koneksi ke Broker sekali saja di awal. Setelah itu, jalur komunikasi (soket TCP) bakal terus terbuka secara otomatis (Keep-Alive). Saat ada perintah masuk, pesan langsung diterima saat itu juga tanpa perlu bongkar-pasang koneksi lagi, jadi pemakaian daya CPU dan memori ESP8266 jauh lebih santai.

3. Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (*1-to-N distribution*).
   Jawab : Jika pakai HTTP yang sifatnya Point-to-Point (1-ke-1), misal ada 100 aplikasi yang mau baca data dari sensor kita, ESP8266 harus ngirim data tersebut satu per satu sebanyak 100 kali. Mikrokontroler kecil ESP8266 jelas tidak akan sanggup dan langsung crash.

   Di MQTT yang pakai konsep Publish-Subscribe, ESP8266 cuma bertugas ngirim paket data cukup 1 kali saja ke Broker. Nanti Broker ini yang akan membagikan pesan tersebut secara serentak ke 100 aplikasi atau perangkat lain yang sudah berlangganan (subscribe) di topik tersebut. ESP8266-nya jadi tidak terbeban sama sekali.

Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):
Sebuah pabrik manufaktur memiliki 3 lini produksi (*line1*, *line2*, *line3*), di mana setiap lini memiliki 4 mesin (*m1*, *m2*, *m3*, *m4*), dan setiap mesin melaporkan metrik `suhu`, `getaran`, dan `status`. Struktur topiknya adalah `pabrik/<line>/<mesin>/<sensor>`. Tentukan ekspresi topik subskripsi (*subscription topic*) yang tepat menggunakan karakter *wildcard* (`+` atau `#`) untuk:

1. Memantau parameter `suhu` dari seluruh mesin di semua lini produksi.
   jawab :
   Ekspresi Topik: **pabrik/+/+/suhu**

   Jadi, karakter wildcard + (single-level) digunakan untuk menggantikan tepat 1 tingkat hierarki. Tanda + pertama menggantikan semua lini produksi (line1, line2, line3), dan tanda + kedua menggantikan semua mesin (m1, m2, m3, m4). Dengan ekspresi ini, hanya akan menangkap data khusus sensor suhu dari mesin mana saja dan lini mana saja.

2. Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin `m1` di lini `line2`.
   jawab :
   Ekspresi Topik: **pabrik/line2/m1/+**

   Jadi, karena lokasi lini (line2) dan mesin (m1) sudah spesifik, hanya perlu memasang wildcard + pada tingkat akhir (<sensor>). Tanda + di posisi terakhir ini akan secara otomatis menangkap semua jenis sensor yang dilaporkan oleh mesin m1, yaitu suhu, getaran, maupun status.

3. Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut.
   jawab :
   Ekspresi Topik: **pabrik/#**

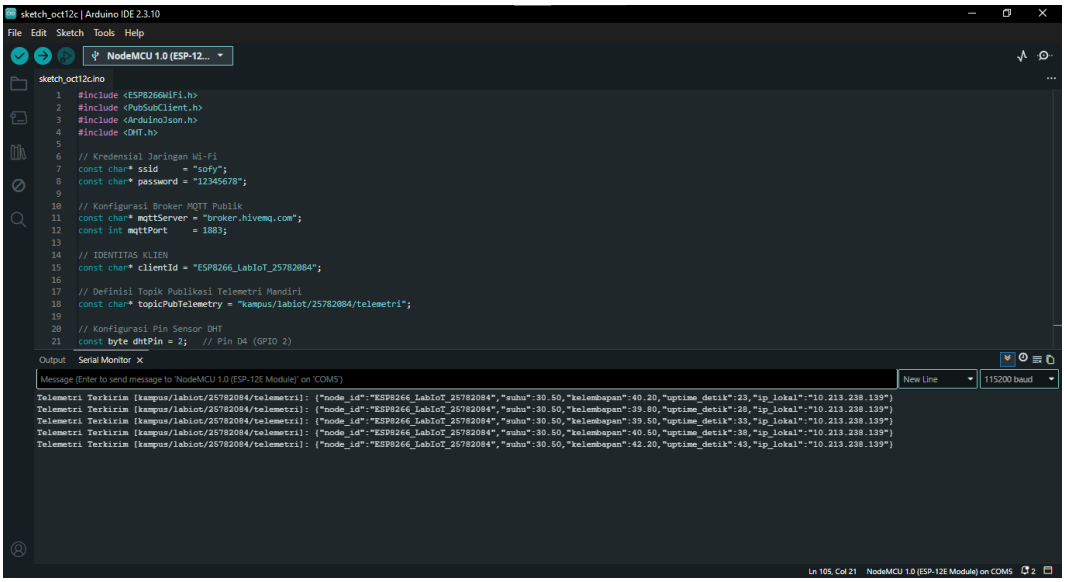
   Jadi, karakter wildcard # (multi-level) untuk menggantikan seluruh sisa tingkat hierarki turunan di bawahnya. Dengan memasang # setelah pabrik/, sistem akan menangkap semua pesan yang berawalan dari kata pabrik, tanpa peduli apa nama lini, mesin, maupun jenis sensornya.

Latihan 3 (Implementasi Serialisasi Data Dinamis):
Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:

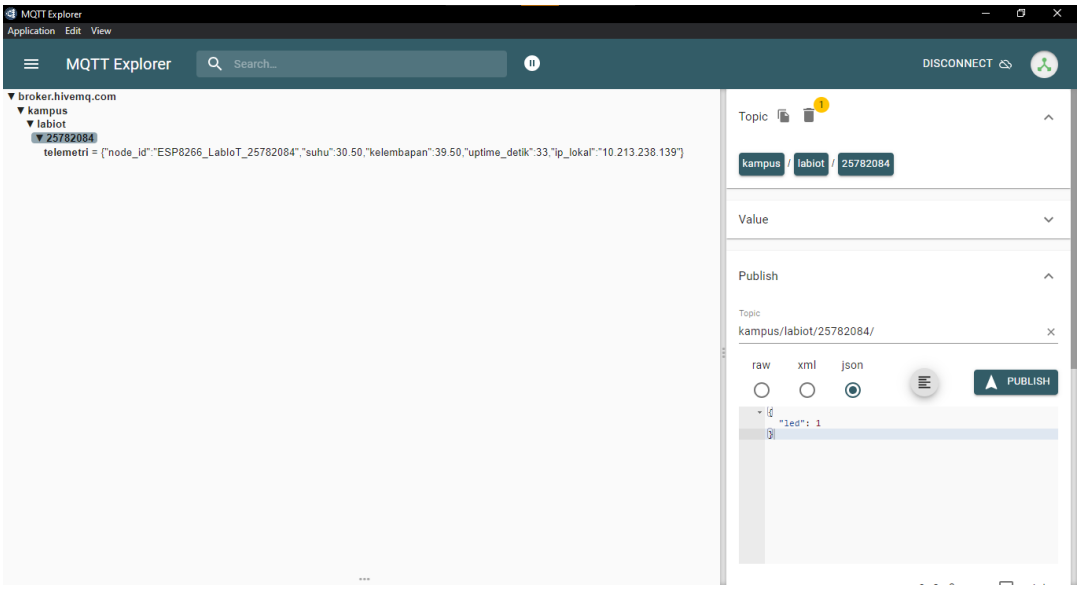
1. `"uptime_detik"`: Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus `millis() / 1000`).
2. `"ip_lokal"`: Alamat IP lokal mikrokontroler dalam format string (`WiFi.localIP().toString()`).
   Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!
   Jawab :
   Ini kode yang ditambahkan :
   doc["uptime_detik"] = millis() / 1000
   doc["ip_lokal"] = WiFi.localIP().toString();

   Jadi , pada latihan disini saya modifikasi kode program JSON dengan menambahkan dua baris instruksi:
   - **doc["uptime_detik"] = millis() / 1000;** yaitu Menghitung durasi aktif perangkat dalam satuan detik.
   - **doc["ip_lokal"] = WiFi.localIP().toString();** yaitu Menangkap alamat IP lokal NodeMCU dari Wi-Fi.

   Menampilkan log pengiriman data telemetri dari NodeMCU ke broker. Terlihat dua parameter baru, "uptime_detik" yang nilainya terus bertambah (23, 28, 33, 38, 43) menandakan durasi sistem aktif, serta "ip_lokal":"10.213.238.139" yang menunjukkan alamat IP perangkat di jaringan Wi-Fi.



Menampilkan penerimaan paket data JSON pada topik kampus/labiot/25782084/telemetri secara real-time di broker. Pada payload yang diterima memuat variabel "uptime_detik":33 dan "ip_lokal":"10.213.238.139", membuktikan data berhasil dikirim dan diterima secara penuh.



Jadi, penambahan instruksi doc["uptime_detik"] dan doc["ip_lokal"] pada struktur JSON berhasil menambah informasi status sistem pada paket telemetri. Jadinya, NodeMCU ESP8266 (Publisher) bisa mengirimkan durasi waktu aktif alat (uptime) dan alamat IP lokal Wi-Fi secara dinamis ke broker, serta berhasil diterima secara utuh dan real-time oleh MQTT Explorer (Subscriber) pada topik kampus/labiot/25782084/telemetri.

Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking): Pada Praktikum 3, di dalam fungsi `loop()` terdapat pemanggilan rutin `client.loop()`.

1. Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah *callback* jika seorang pengembang menyisipkan instruksi `delay(6000)` di dalam fungsi `loop()`?

   Jawab : Penyisipan instruksi delay(6000) akan memblokir (blocking) eksekusi eksekusi program mikrokontroler secara total selama 6 detik pada setiap siklus. Konsekuensi teknis yang terjadi meliputi:

   - **Penurunan Responsivitas Callback:**
     Fungsi client.loop() untuk memeriksa antrean paket TCP/MQTT yang masuk dari broker. Jika terhalang delay(6000), fungsi

client.loop() tidak dipanggil selama 6 detik tersebut. Akibatnya, instruksi perintah kontrol LED yang dipublikasikan oleh Publisher akan tertahan (lagging/delay) dan baru diproses oleh fungsi callback setelah jeda 6 detik berlalu.

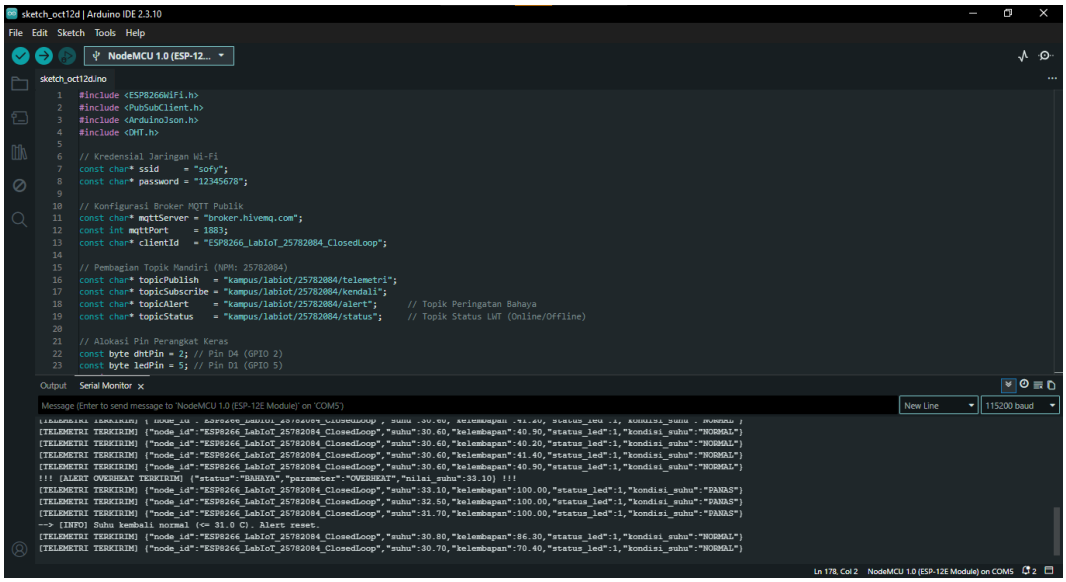
- **Risiko Pemutusan Koneksi (Keep-Alive Timeout):**
  MQTT broker mengandalkan pingreq/pingresp (keep-alive) untuk memantau status klien. Jika jeda delay() terlalu lama melebihi ambang batas keep-alive, broker dapat menganggap NodeMCU telah terputus (offline/timeout) dan memutuskan koneksi TCP secara sepihak.

2. Jelaskan bagaimana fungsi `reconnect()` bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat!
   Jawab : Fungsi reconnect() bekerja secara otomatis untuk menjamin keandalan sistem (auto-recovery) dengan alur sebagai berikut:
   - **Pemeriksaan Status Koneksi (!client.connected()):**
     Pada setiap siklus loop(), program akan memeriksa status koneksi jaringan dan broker. Jika router Wi-Fi terputus atau koneksi ke HiveMQ terhenti, pemanggilan client.connected() akan bernilai false.
   - **Pengulangan Koneksi Ulang (Retry Loop):**
     Fungsi reconnect() menjalankan perulangan while (!client.connected()) untuk mencoba menyambungkan kembali mikrokontroler ke broker menggunakan client.connect(clientId).
   - **Pendaftaran Ulang Subskripsi (Resubscription):**
     Saat koneksi TCP/MQTT berhasil dipulihkan, baris client.subscribe(topicSubscribe) langsung dieksekusi kembali secara otomatis. Hal ini memastikan NodeMCU mendaftarkan ulang perannya sebagai Subscriber pada topik kendali agar tidak kehilangan kemampuan menerima instruksi kontrol setelah jaringan pulih.

Tugas: Sistem Telemetri & Kendali Closed-Loop Industri

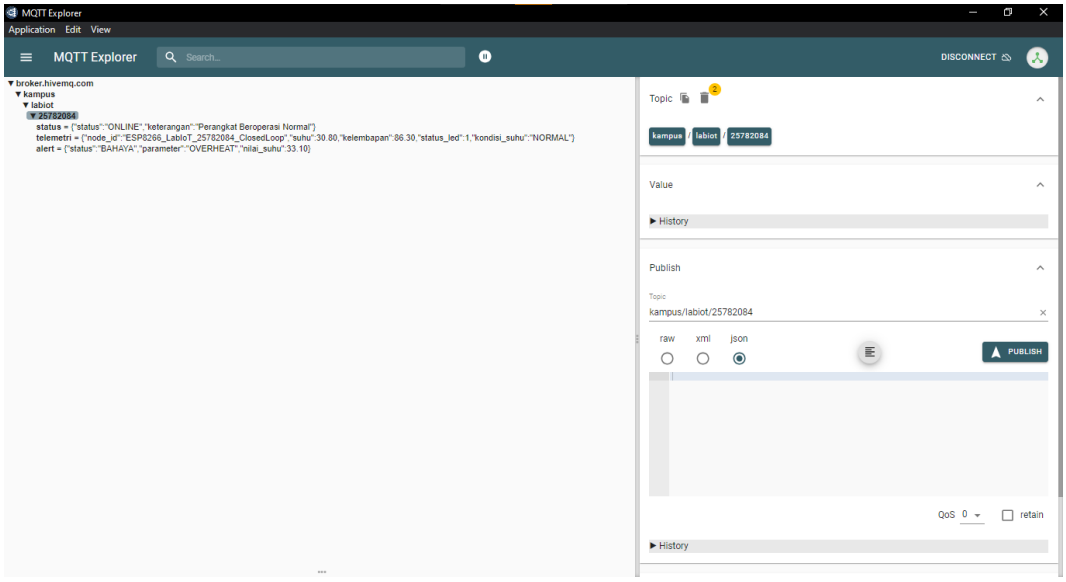
Jadi setelah dijalankan atau run, serial monitor menampilkan log dari program ESP8266_LabIoT_25782084_ClosedLoop. Terlihat saat kondisi normal Kurang dari atau sama dengan 31.0 celsius, telemetri dikirim dengan "kondisi_suhu":"NORMAL".

Ketika suhu melampaui 31.0 celsius , sistem langsung memicu edge safety engine dengan menampilkan sinyal darurat !!! [ALERT OVERHEAT TERKIRIM] {"status":"BAHAYA","parameter":"OVERHEAT","nilai_suhu":33.10} !!! ke topik alert serta mengubah status telemetri menjadi "PANAS". Ketika suhu kembali aman, tercetak log --> [INFO] Suhu kembali normal (<= 31.0 C). Alert reset..

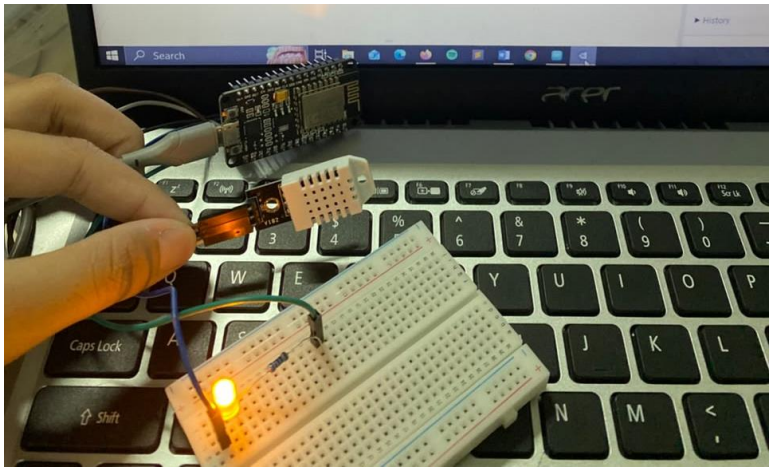


MQTT Explorer Terlihat tiga topik aktif utama:

- **status**: Menampilkan pesan status aktif (LWT Online) {"status":"ONLINE","keterangan":"Perangkat Beroperasi Normal"}.
- **telemetri**: Menampilkan paket telemetri dinamis ter-update {"node_id":"...","suhu":30.80,"kelembapan":86.30,"status_led":1,"kondisi_suhu":"NORMAL"}.
- **alert**: Menampilkan riwayat pesan bahaya saat terjadi overheat {"status":"BAHAYA","parameter":"OVERHEAT","nilai_suhu":33.10}



Jadi, saat sensor DHT22 mendeteksi peningkatan suhu lingkungan di atas 31.0 celsius NodeMCU secara otomatis menyalakan LED sebagai tindakan proteksi darurat (automated actuation).



Jadi pada tugas ini saya sudah melalukan sistem proteksi Closed-Loop berbasis MQTT berhasil diimplementasikan. NodeMCU ESP8266 mampu bertindak di tingkat edge dengan mengklasifikasikan kondisi suhu (NORMAL/PANAS), mengirimkan sinyal peringatan bahaya secara mandiri ke topik alert saat terjadi overheat, serta melaporkan status keandalan koneksi secara real-time melalui mekanisme Last Will and Testament (LWT) pada topik status.